

UNIT-3 HTTP and ASP.NET Core

To communicate with a web server, the client, makes calls over the network using HTTP. A client makes an HTTP request for a resource, and the server, sends back an HTTP response.

How does an HTTP web request work?

1. User requests a web page by a URL.

`http://notejunction.com/page.html`

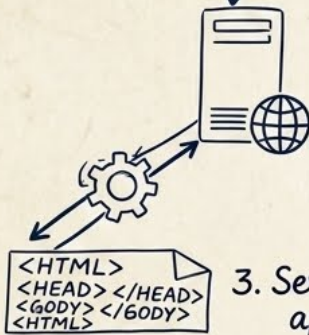
5. Browser renders HTML on page.

`http://notejunction.com/page.html`

Welcome to Website!

2. Browser sends HTTP request to server.

HTTP request



4. Server sends HTML in HTTP response back to browser.

HTTP response

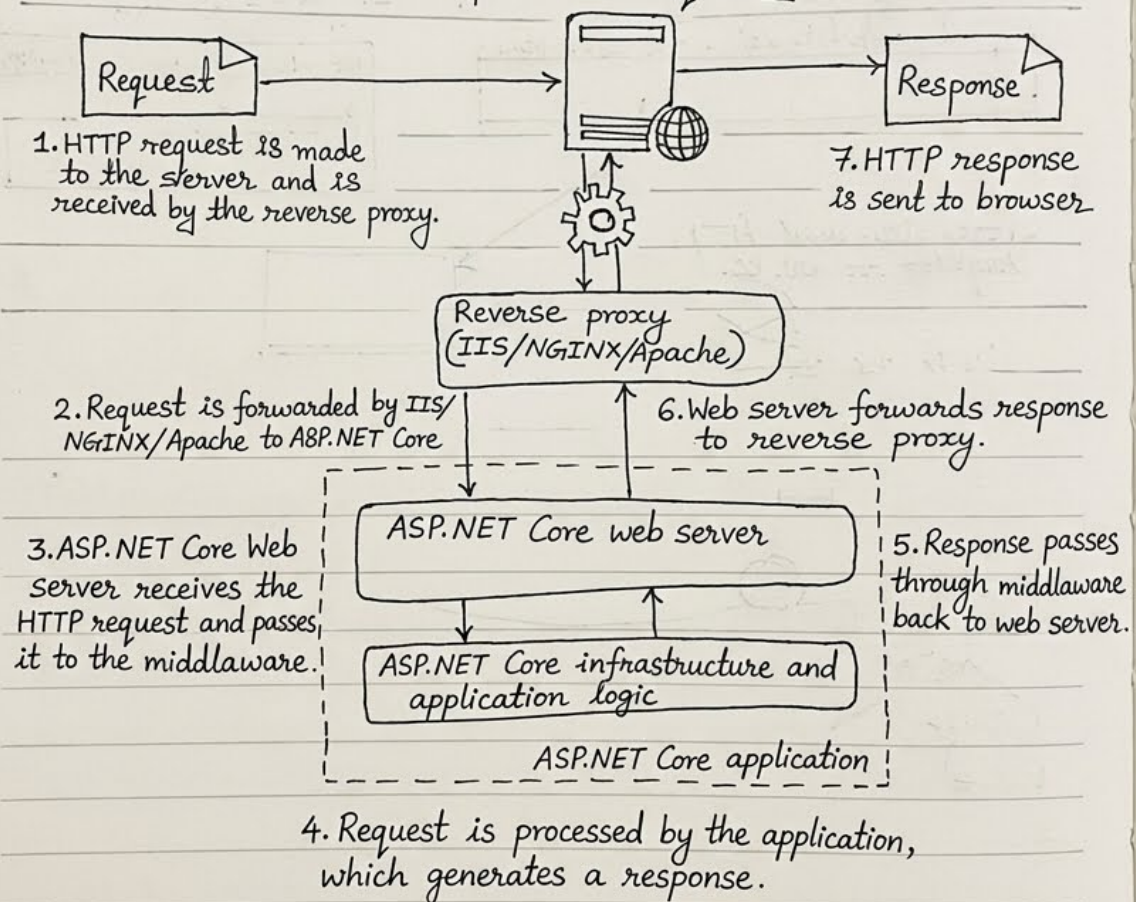
3. Server Interprets request and generates appropriate HTML.

```
<HTML>
<HEAD> </HEAD>
<BODY> </BODY>
<HTML>
```

→ The user starts by requesting a web page, which causes an HTTP request to be sent to the server. The server interprets the request, generates the necessary HTML, and sends it back on an HTTP response. The browser can then display the web page.

- Once the server receives the request, it will check that it makes sense, and if it does, will generate an HTTP response. Depending on the request, this response could be a web page, an image, a JavaScript file, or a simple acknowledgment.
- As soon as the user's browser begins receiving the HTTP response, it can start displaying content on the screen, but the HTML page may also reference other pages and links on the server.

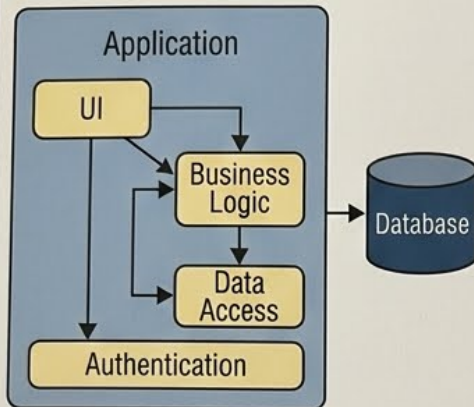
How does ASP.NET Core process a request?



→ A request is received from a browser at the reverse proxy, which passes the request to the ASP.NET Core application, which runs a self-hosted web server.

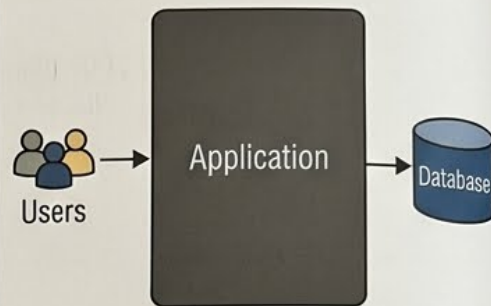
Page 18: Common Web Application Architectures

1 Monolithic Application



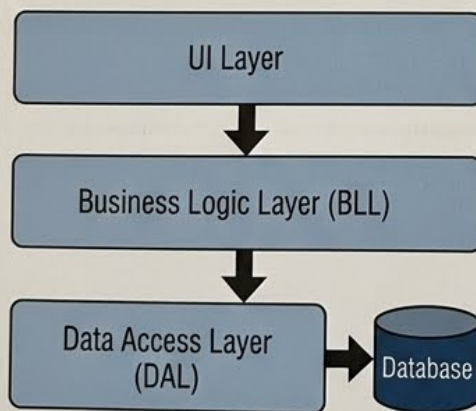
Entire logic is within one process, deployed as one unit.

2 All-in-One Application



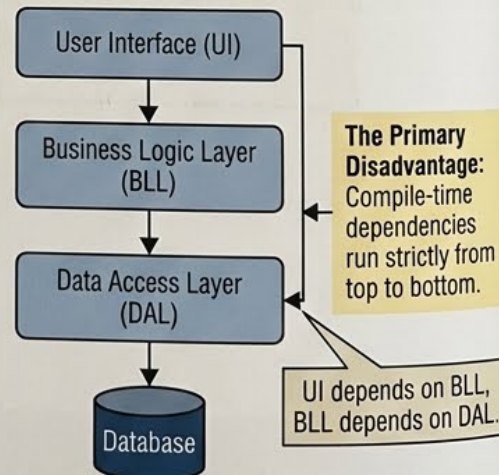
Single project, single assembly, single unit.

3 Layered Architecture

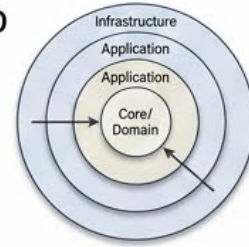


Breaks up application; enforces communications restrictions between layers.

4 Traditional 'N-Layer' Architecture Applications



CLEAN ARCHITECTURE: Applications that adhere to Inversion Principle, as well as Domain-Driven Design (DDD) principles, are known as having Clean Architecture. It places the central business logic and application model at the core of the application.



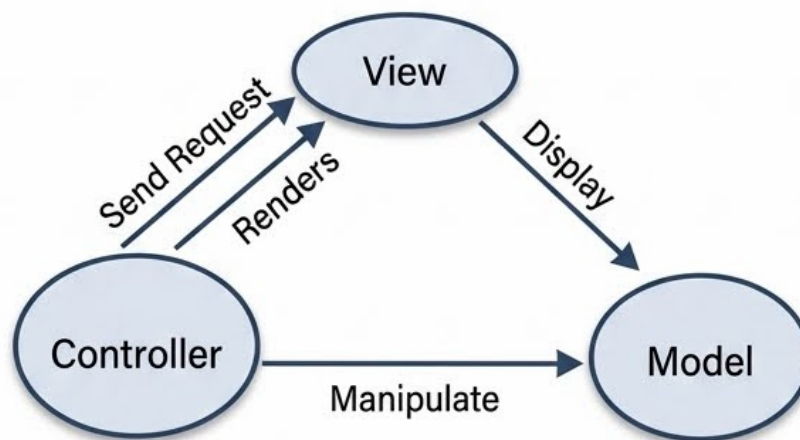
MVC PATTERN / ARCHITECTURE

MVC stands for Model, View, and Controller. MVC separates an application into three distinct components.

Model: It represents the data shape. In C#, a class describes is used to describe a model. Jiael. Model objects store data from objects store and represent retrieved data.

View: The user interface. A user interface. View display Model data to the user and also enables them to modify them. View in ASP.NET MVC, it is HTML, CSS, and special syntax (Razor syntax) that makes it easy to communicate with the model and the controller.

Controller: The request handler. It is typically, the user interacts with the View which raises an HTTP request. The Controller processes the request and returns the appropriate View as a response. Controller is the request handler.



ASP.NET Core Architecture Overview

The ideology behind ASP.NET Core in general is to lay out web logic, infrastructure, and core components from each other in order to provide a more development-friendly environment. In ASP.NET Core, the main business logic and UI logic are encapsulated in ASP.NET Core Web App layer, while the database access layer, cache services, and web API services are encapsulated in infrastructure layer and common utilities objects, interfaces and reusable business services are encapsulated as micro-services in application core layer.

ASP.NET Core creates necessary pre-defined “N” tier architecture for developers automatically which saves time and effort. It has the benefit of a pre-built architectural framework that eases out tier deployment of the project along with providing pre-build Single Page Application (SPA) design patterns, Razor pages design and traditional MVC design pattern.

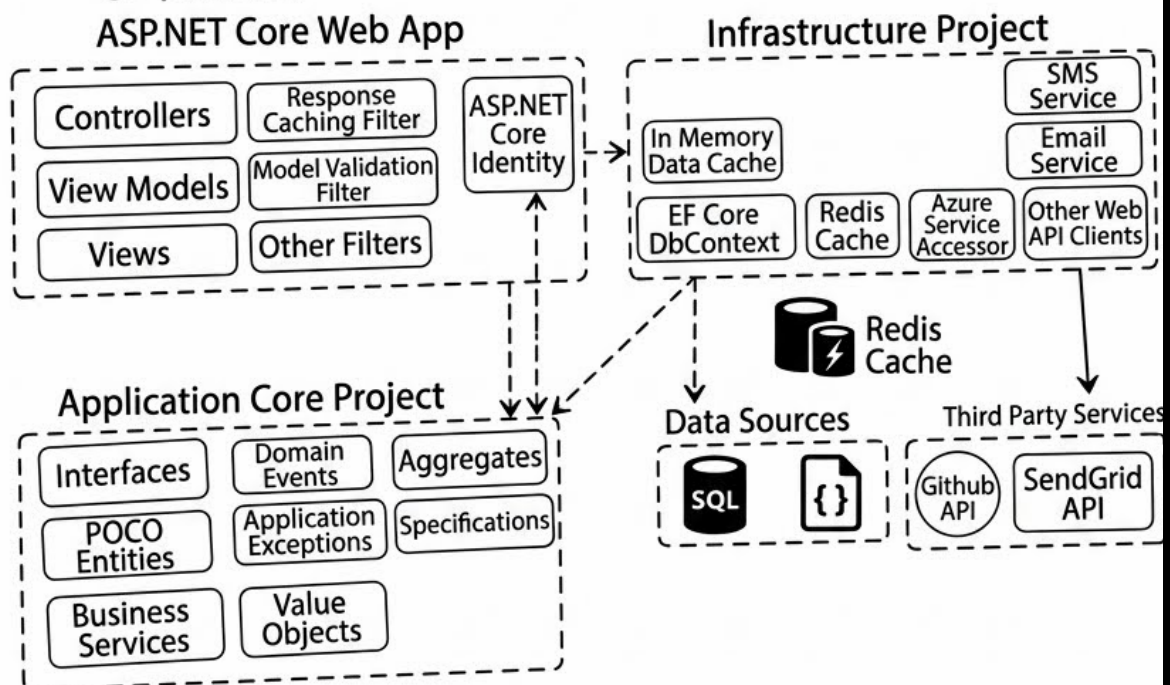


Fig: ASP.NET Core Architecture Overview.

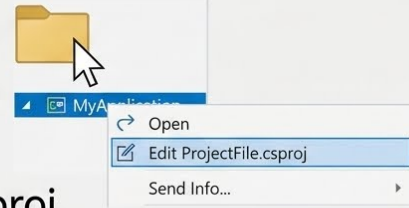
Projects and Conventions in .NET

- 1 Visual Studio now uses **.csproj** file to manage projects.

We can **edit** the **.csproj** settings by:

We can **edit** the **.csproj** settings by:

- → right click on the project.
- → select **Edit** <project-name.csproj>



- 2 The csproj file includes settings related to targeted .NET frameworks, project folders, NutGet package references etc.

Target Framework	→	<code><.xml xmlns="1.0" ennding="uf-8"></code>
		<code><TargetFramework>net8.0</TargetFramework></TargetFramework></Project></code>
Project Folders	→	<code><Project Item></code>
		<code><Compile Include="Models***" /></code>
NuGet Package References	→	<code><Packages References></code>
		<code><PackageReference Include="Newtonsoft.Json" Version="13.0.3" /></code>
		<code></Package></code>
		<code></Project></code>